Trabalho 1 – Concorrência e Sincronização: O Problema dos Filósofos Jantando

Fundamentos de Processamento Paralelo e Distribuído – Prof. De Rose

Grupo	R
Integrantes	*Nome A* · *Nome B* · *Nome C*
Repositório Git	*link do repositório* (pasta `cmd/`, uma subpasta por versão)

Descrição das soluções e mecanismos de sincronização

As quatro versões partem do exemplo sequencial do Moodle (`filo_seq.go`) e mudam só a forma de pegar os garfos. Todas usam 5 filósofos, 20 ciclos cada, 20 ms para pensar e 30 ms para comer. Cada ciclo é: pensar → pegar os garfos → comer → soltar os garfos.

Versão	Como funciona	Mecanismo	Implementação em Go
V0 – Sequencial	Um laço chama um filósofo de cada vez; ninguém disputa garfo	Nenhum (execução sequencial)	`Philosopher(j, i)` chamado dentro de dois `for`
V1 – Com deadlock	Cada filósofo é uma goroutine; todos pegam o garfo esquerdo e depois o direito	Exclusão mútua por garfo	Um `sync.Mutex` por garfo; `sync.WaitGroup` espera as goroutines
V2 – Sincronização excessiva	Igual à V1, mas um "garçom" só deixa um filósofo comer por vez	Semáforo binário global + mutex por garfo	Go não tem semáforo nativo: canal com buffer 1 (`chan struct{}`); enviar = P, receber = V
V3 – Concorrente	Igual à V1, mas todos pegam primeiro o garfo de menor número	Ordenação global dos recursos + mutex por garfo	`sync.Mutex` por garfo; o F5 troca a ordem (G0 antes de G4)

As métricas usam um `sync.Mutex` próprio, porque os 5 filósofos atualizam os mesmos contadores ao mesmo tempo. O tempo é medido com `time.Now()` e `time.Since()`.

Análise comparativa [PROVISÓRIO]

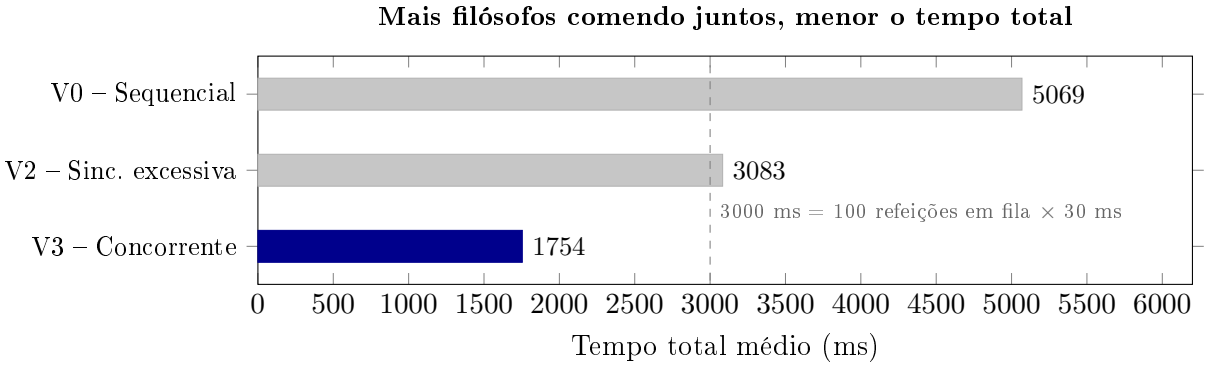
A V3 foi a mais rápida (1,75 s em média), 2,9 vezes mais rápida que a V0 (5,07 s), porque deixa 2 filósofos comerem ao mesmo tempo. A V2 não trava, mas o garçom limita a mesa a 1 filósofo comendo por vez. Por isso ela leva 3,08 s e tem a maior espera pelos garfos: 100 ms em média, contra 23 ms na V3.

- **V0:** tempo total = soma de todos os tempos ($20 \times 5 \times 50\,\text{ms} = 5\,\text{s}$). Espera zero, porque não há disputa.
- **V1:** travou em 4 de 5 execuções, logo no 1.º ciclo. Como não há garantia de que termine, não serve como solução.
- **V2:** é correta e 1,6 vez mais rápida que a V0, porque os filósofos pensam em paralelo. Mas as refeições ficam em fila: $100 \times 30\,\text{ms} = 3\,\text{s}$, praticamente o tempo total medido.
- **V3:** é correta e tem o maior grau de concorrência possível com 5 garfos (2 comendo juntos). Ganha da V2 em tempo total e em espera.

Resultados experimentais [PROVISÓRIO]

Média de 5 execuções de cada versão; entre parênteses, o desvio padrão. *Números de uma máquina de teste com 2 núcleos: substituir pelos do notebook do grupo.*

Versão	Tempo total (ms)	Espera média (ms)	Maior espera (ms)	Máx. comendo juntos	Ciclos por filósofo
V0 – Sequencial	5069 (±2,5)	0,0	0,0	1	20 em todos
V1 – Deadlock	travou em 4 de 5 execuções (0 ciclos concluídos nas que travaram)				
V2 – Sinc. excessiva	**3083 (±2,5)**	**100,2 (±0,1)**	**122,1**	**1**	20 em todos
V3 – Concorrente	1754 (±51)	23,4 (±0,5)	72,1	2	20 em todos



V0 e V2: 1 filósofo comendo por vez. V3: até 2 comendo juntos. A linha tracejada mostra por que a V2 não fica abaixo de 3 s: as 100 refeições acontecem uma de cada vez.

Evidência do deadlock da V1

Saída real de uma execução que travou. Cada coluna é um filósofo: `T` = pensando, `G` = segurando um garfo, `E` = comendo.

```
 [P1]   [P2]   [P3]   [P4]   [P5]
                      T1
 T1
        T1
        G1
               G2
                             G4
                      G3
 G0
fatal error: all goroutines are
             asleep - deadlock!
```

Filósofo	Segura	Espera	Está com
P1	G0	G1	P2
P2	G1	G2	P3
P3	G2	G3	P4
P4	G3	G4	P5
P5	G4	G0	P1

Nenhum filósofo chegou a comer: todos pegaram o garfo esquerdo e ficaram esperando o direito. O runtime do Go detectou que todas as goroutines estavam bloqueadas e encerrou o programa.

Capturas de tela

Inserir aqui os prints da V0, V2 e V3 terminando e da V1 travada, salvos em `resultados/prints/`. Para isso, acrescente ao preâmbulo:

```
\usepackage{graphicx}
```

e, aqui, uma linha por print:

```
\includegraphics[width=0.48\textwidth]{../resultados/prints/v3.png}
```

Conclusão

Mais sincronização não significou mais desempenho. A V2, a mais sincronizada, é correta, mas levou 3083 ms. A V3 sincroniza só o necessário e levou 1754 ms, 43 % menos. O que reduziu o tempo total foi o grau de concorrência: 1 filósofo comendo por vez na V0 e na V2, 2 na V3.

1. Como funciona cada solução?

A V0 executa um filósofo por vez, em laço. Na V1, na V2 e na V3, cada filósofo é uma goroutine que repete 20 vezes: pensar, pegar os dois garfos, comer e soltar. As três diferem só na forma de pegar os garfos: na V1, esquerdo e depois direito; na V2, pede antes a permissão do garçom; na V3, primeiro o garfo de menor número.

2. Quais mecanismos de sincronização são utilizados?

Exclusão mútua (um lock por garfo) na V1, na V2 e na V3. Na V2, também um semáforo binário global. Na V3, ordenação global dos recursos. Em todas, uma barreira de término para o `main` esperar os filósofos.

3. Como esses mecanismos são implementados em Go?

O lock de cada garfo é um `sync.Mutex` (`Lock`/`Unlock`). A barreira é um `sync.WaitGroup` (`Add`, `Done`, `Wait`). Go não tem semáforo na biblioteca padrão, então o semáforo da V2 foi adaptado com um canal com buffer de 1: `waiter <- struct{}{}` é o P (wait) e `<-waiter` é o V (signal). A ordenação da V3 é só um `if` que troca a ordem dos dois garfos.

4. Por que ocorre deadlock na V1?

Porque as quatro condições de Coffman estão presentes ao mesmo tempo:

- **Exclusão mútua:** um mutex só pode estar com uma goroutine.
- **Segura e espera:** cada filósofo segura o garfo esquerdo enquanto espera o direito.
- **Sem preempção:** ninguém tira o garfo de outro; só o dono faz `Unlock`.
- **Espera circular:** P1 espera P2, que espera P3, ..., que espera P1 (tabela da página 2).

Isso aconteceu em 4 das 5 execuções, sempre no 1.º ciclo, porque todos pensam o mesmo tempo e chegam juntos aos garfos.

5. Como V2 e V3 evitam deadlock?

A V2 quebra o "segura e espera": só quem tem a permissão do garçom pega garfos, então nunca encontra um garfo ocupado. A V3 quebra a espera circular: como todos pegam primeiro o garfo de menor número, um ciclo exigiria $G_a < G_b < \ldots < G_a$, o que é impossível. Na prática, P1 e P5 disputam o G0 como primeiro garfo, e quem perde espera sem segurar nada.

6. Existe possibilidade de starvation? Justifique.

Na V0, não: a ordem é fixa. Na V2 e na V3, é teoricamente possível, porque nada garante a vez de um filósofo. Nos experimentos ela não ocorreu: todos completaram 20 ciclos e a maior espera foi 122 ms na V2 e 72 ms na V3 (menos de 3 refeições). Isso acontece porque o `sync.Mutex` do Go passa a atender em ordem de chegada quando uma goroutine espera mais de 1 ms, e porque cada filósofo pensa entre as refeições, dando vez aos vizinhos.

7. Qual solução apresentou maior tempo total de execução?

A V0, com 5069 ms, por ser sequencial. Entre as corretas e concorrentes, a V2 (3083 ms).

8. Qual solução apresentou maior tempo de espera pelos recursos?

A V2: 100,2 ms em média e 122,1 ms no pior caso, contra 23,4 ms e 72,1 ms na V3. A V1 fica de fora, porque sua espera é infinita quando trava.

9. Quantos filósofos conseguiram comer simultaneamente em cada solução?

V0: 1. V1: 0 nas execuções que travaram e 1 na que terminou. V2: 1. V3: 2, o máximo possível com 5 garfos.

10. Qual foi o impacto da sincronização excessiva da V2?

A V2 transforma as 100 refeições em uma fila: $100 \times 30\,\text{ms} = 3000\,\text{ms}$, quase todo o tempo medido (3083 ms). Comparada com a V3, foi 76 % mais lenta e teve espera média 4,3 vezes maior, sem nenhum ganho de corretude.

11. A solução com maior sincronização foi a de melhor desempenho?

Não. A V2 tem mais sincronização que a V3 (um semáforo global além dos mutexes) e foi pior em todas as métricas de tempo: 3083 ms contra 1754 ms no total, e 100,2 ms contra 23,4 ms de espera média. Sincronização garante corretude, mas cada trava a mais tira concorrência.

12. Qual é a relação entre o grau de concorrência e o tempo total?

Quanto maior o grau de concorrência, menor o tempo total. Com 1 filósofo ativo por vez (V0), 5069 ms. Com pensamento em paralelo, mas refeições em fila (V2), 3083 ms. Com até 2 comendo juntos (V3), 1754 ms.

Ambiente de execução

Linguagem e versão	Go *x.y.z* (saída de `go version`)
Sistema operacional	*Windows 11 ...*
Processador	*modelo* (Gerenciador de Tarefas → Desempenho → CPU)
Núcleos disponíveis	*físicos / lógicos*
Parâmetros	5 filósofos, 20 ciclos, pensar 20 ms, comer 30 ms (iguais nas 4 versões)

```
go build -o bin/v3-concorrente ./cmd/v3-concorrente  # compila
./bin/v3-concorrente                                 # executa
go run ./cmd/v1-deadlock                             # compila e executa
go vet ./... && gofmt -l .                           # confere o codigo
./scripts/executar.sh                                # 5 execucoes de cada
```